Hibernate with JPA

Software Developer Interview — Questions & Answers Guide

Core Concepts | Architecture | JPQL | Caching | Lifecycle | Relationships

Section 1: JDBC Limitations & Introduction to Hibernate

Q1. What are the key limitations of JDBC that led to the creation of Hibernate?

Answer:

- Java developers are forced to know SQL — not object-oriented.
- Manual table creation is mandatory; no automatic schema generation.
- Exception handling is compulsory — all exceptions are checked (ClassNotFoundException, SQLException).
- No cache memory support — every operation hits the database directly.
- Manual ResultSet processing is required to extract data.
- Object-relational mismatch (impedance mismatch) — Java objects cannot be mapped directly to table records.
- Boilerplate code is repeated throughout the application.
- Developers must know database/table properties and SQL syntax for each database vendor.
- SQL syntax differs between database vendors — not portable.

Q2. What is a Framework? How does it differ from a library?

Answer:

A Framework is an application used to develop other applications. It is a collection of pre-built libraries, classes, and interfaces.

- Main purpose: Eliminate boilerplate/repetitive code.
- Speeds up development by providing ready-made components.
- Example: Hibernate is a Java framework to work with database applications.

Key Difference from a Library:

- Library: You call the code (you control the flow).
- Framework: The framework calls your code (inversion of control).

Q3. What is Hibernate ORM Framework? Why is it widely used?

Answer:

Hibernate is an open-source Java-based ORM (Object Relational Mapping) tool/framework that acts as a bridge between the object-oriented Java model and relational databases.

- It is used as an alternative to JDBC.
- It is common/compatible with all DBMS vendors.
- Maps Java classes to database tables, Java datatypes to SQL datatypes, and Java variables to table columns.
- Has built-in mechanisms for driver loading, connection creation, statement creation.
- Automatically creates tables from entity classes.
- Automatically handles checked exceptions.
- Supports HQL/JPQL — database-platform-independent query language.
- Widely used because it is open-source and rich in features.

Q4. What is ORM? What is its main purpose?

Answer:
ORM stands for Object Relational Mapping. It is a technique that maps object-oriented data representations to relational database data.

- Main purpose: DATA Persistence — storing the state of objects outside heap memory for future access.
- Overcomes JDBC limitations, including the impedance mismatch problem.
- Maps Java data types with DB data types, and object states (non-static variables) with table columns.

Three ORM Frameworks in Java:

- Hibernate
- iBatis
- Toplink / EclipseLink

ORM frameworks depend on POJO (Plain Old Java Object) classes to keep them lightweight.

Section 2: JPA — Java Persistence API

Q5. What is JPA? Why was it introduced?

Answer:
JPA stands for Java Persistence API. It is a set of specifications (abstract functionality) that defines how Java objects can be mapped to relational database tables (ORM).

- JPA itself has no implementation — it is a blueprint/specification.
- It is part of the javax.persistence package.
- Introduced to solve the problem of vendor lock-in: if a developer wanted to switch from Hibernate to iBatis, they had to rewrite the entire application.
- JPA provides a common specification that all ORM frameworks implement, making it easy to switch providers.

JPA consists of:

- EntityManagerFactory interface
- EntityManager interface
- EntityTransaction interface
- Query interface
- Persistence class

Q6. What is the difference between JPA and Hibernate?

Answer:

JPA:

- A specification (set of rules/interfaces) — no implementation of its own.
- Defined by javax.persistence package.
- Acts as a standard API for ORM frameworks.
- Cannot be used directly — needs an implementation.

Hibernate:

- A concrete implementation of the JPA specification.
- An open-source ORM framework.
- Called 'Hibernate JPA' when used as the JPA provider.
- Provides additional features beyond JPA (e.g., caching, HQL).

Analogy: JPA is like an interface; Hibernate is the class that implements it.

Q7. What is Data Persistence?

Answer:

Data Persistence means storing the state of an object outside the scope of JVM heap memory for future access.

- Data persists even after the program stops — it is not lost.
- Programmer can store object states in:

- Files (Excel, CSV)
- Database (Relational/NoSQL)

- Hibernate/JPA's main purpose is to facilitate Data Persistence.

Q8. How does Hibernate interact with the database internally?

Answer:

Hibernate does NOT directly interact with the database. It uses JDBC API internally.

Flow of communication:

- Java Application → writes code using JPA API
- JPA API → communicates with Hibernate Framework
- Hibernate Framework → uses JDBC API and JDBC Driver internally

- JDBC Driver → communicates with the Database

Key point: In Java, JDBC API is the only standard to communicate with databases. Hibernate is a higher-level abstraction on top of JDBC.

Section 3: Entity Class & Annotations

Q9. What is an Entity Class in Hibernate/JPA?

Answer:

An Entity Class is a Java class annotated with @Entity that is mapped to a database table.

- The class name maps to the table name (by default).
- Variables in the class represent the columns of the table.
- When an object of the entity class is created and persisted, data is stored in the mapped table.

Example:

```
@Entity
public class Employee {
    @Id
    private int id;
    private String name;
}
```

Note: All annotations must be imported from javax.persistence package.

Q10. What are the rules to create a valid Entity Class in Hibernate?

Answer:

- @Entity must be placed on a public, non-abstract (concrete) class only.
- Entity class must have one Primary Key — annotated with @Id.
- Entity class must have a public no-args constructor (required for fetch, update, delete).
- Every variable must have public getter and setter methods.
- Entity class should implement Serializable (optional but recommended).
- All annotations must be imported from javax.persistence package.

Q11. What is the purpose of @Id, @Column, @Table annotations?

Answer:

@Id:

- Marks a variable as the primary key of the entity/table.
- Required to uniquely identify each record in the table.

- Mandatory for fetch, update, and delete operations.

@Column:
- Maps a Java field to a specific database column.
- Attributes: name (column name), length (size), nullable (NOT NULL), unique (unique constraint).

@Table:
- Specifies the database table name for the entity.
- Attributes: name, catalog, schema, unique constraints.

```
@Entity
@Table(name = "EMPLOYEE")
public class Employee { ... }
```

Q12. What is @GeneratedValue? What are the 4 generation strategies?

Answer:

@GeneratedValue defines how the primary key is automatically generated by Hibernate.

1. AUTO (Default):
- Hibernate decides the strategy based on the database.
- MySQL/PostgreSQL → behaves like a sequence.
- Creates a sequence table alongside the entity table.

2. IDENTITY:
- Uses database auto-increment (1, 2, 3, 4…).
- No separate sequence table created.
- Simplest approach for small applications.

3. SEQUENCE:
- Uses a database sequence object.
- Best for Oracle, PostgreSQL — high-performance batch inserts.
- Can be customized with @SequenceGenerator.

4. TABLE:
- Uses a special table (hibernate_sequences) to generate keys.
- Works across all databases.
- Use when DB doesn't support sequences and cross-DB portability is needed.

Q13. What are @CreationTimestamp and @UpdateTimestamp annotations?

Answer:

@CreationTimestamp:
- Automatically sets the field value to the current date/time when the entity is first inserted into the database.

- Does NOT change on subsequent updates.

@UpdateTimestamp:
- Automatically updates the field with the current date/time whenever the entity is updated.
- Also sets the value when the entity is first inserted.

Both are Hibernate-specific annotations useful for auditing — tracking when records are created or last modified.

Setion 4: EntityManagerFactory, EntityManager & EntityTransaction

Q14. What is EntityManagerFactory? What is its role?

Answer:
EntityManagerFactory (EMF) is an interface in javax.persistence package used to:
- Create EntityManager instances for interacting with a specific database.
- Manage multiple EntityManager instances when multiple connections are needed.
- Read configuration from persistence.xml to set up the persistence unit.

- One EMF per database / per persistence unit.
- Created using: Persistence.createEntityManagerFactory("persistence-unit-name")
- EMF itself does not directly connect to the database — it creates EntityManagers that handle actual persistence.

Working steps of EMF:
- Step 1: Reads persistence.xml and loads configuration.
- Step 2: Sets up the Persistence Provider (Hibernate), initializes connection pool, entity mappings, and dialect.
- Step 3: Returns a ready-to-use EntityManagerFactory object.

Q15. What is EntityManager? What methods does it provide?

Answer:
EntityManager (EM) is an interface in javax.persistence package. It is the primary interface used by the application to interact with the persistence context.
- Persistence context = a temporary in-memory set of managed entity instances.
- Created using: EntityManager em = emf.createEntityManager();

Key Methods:
- persist(object) — INSERT: saves a new entity to the database.
- merge(object) — UPDATE: updates an existing entity in the database.
- remove(object) — DELETE: deletes an entity from the database.
- find(Class, primaryKey) — SELECT: fetches an entity by its primary key.

- createQuery(String) — Creates a JPQL query object.

Multiple EM instances can be created from the same EMF to prevent data inconsistency.

Q16. What is EntityTransaction? Why is it needed?

Answer:
EntityTransaction is an interface in javax.persistence package used to control database transactions for DML operations (INSERT, UPDATE, DELETE).

Key Methods:
- begin() — Starts the transaction.
- commit() — Saves/commits all changes to the database.
- rollback() — Reverts changes if something fails.

Why needed?
- A transaction is a group of operations that either ALL succeed or ALL fail.
- Without a transaction, partial updates could corrupt data.
- Obtained via: EntityTransaction et = em.getTransaction();

```
et.begin();
em.persist(employee);
et.commit();
```

Section 5: JPQL — Java Persistence Query Language

Q17. What is JPQL? How is it different from SQL?

Answer:
JPQL (Java Persistence Query Language) is a platform-independent, object-oriented query language defined as part of the JPA specification.
- Used to perform custom operations on the database that are not tied to a primary key.
- Works with entity class names and properties — not table names and column names.

JPQL vs SQL:
- SQL uses table names: SELECT * FROM employee
- JPQL uses class names: SELECT e FROM Employee e
- JPQL does NOT support INSERT queries.
- JPQL does NOT support asterisk (*) for SELECT all — must reference the entity alias.
- JPQL is compatible with all ORM tools and databases.

Created via: Query query = em.createQuery(jpqlString);

Q18. What are the 4 methods of the JPQL Query interface?

Answer:

1. executeUpdate():
- Executes UPDATE and DELETE JPQL queries.
- Return type: int (number of rows affected).

2. getSingleResult():
- Executes a SELECT query returning exactly one record.
- Return type: Object (requires downcasting).
- Throws NoResultException if no data is found.

3. getResultList():
- Executes a SELECT query returning multiple records.
- Return type: List.

4. setParameter():
- Assigns values to placeholders in JPQL queries.
- Prevents SQL injection and supports dynamic input.

Q19. What are Positional and Named Parameters in JPQL?

Answer:

JPQL Query Parameters are placeholders that allow dynamic values in queries, making them flexible, reusable, and secure.

1. Positional Parameters:
- Denoted by ? followed by a number (e.g., ?1, ?2).
- Referenced by their position in the query.

```
SELECT e FROM Employee e WHERE e.name = ?1
```

2. Named Parameters:
- Denoted by : followed by a name (e.g., :name, :salary).
- Referenced by descriptive name — more readable.

```
SELECT e FROM Employee e WHERE e.name = :name
```

Both types use setParameter() to bind values. Named parameters are recommended for clarity and maintainability.

Q20. Why doesn't JPQL support INSERT? What are valid JPQL SELECT forms?

Answer:

JPQL does not support INSERT because JPA handles object creation and persistence through em.persist() — the ORM layer manages inserts automatically.

Valid JPQL SELECT forms:

- SELECT e1 FROM Emp e1 ✓ (valid)
- FROM Emp e1 ✓ (valid shorthand)

Invalid JPQL SELECT forms:

- SELECT * FROM Emp e1 ✗ (asterisk not allowed)
- SELECT e1 * FROM Emp e1 ✗ (invalid syntax)
- SELECT e1 FROM Emp ✗ (alias required)
- FROM Emp ✗ (alias required)

Section 6: Hibernate Lifecycle & Caching

Q21. What are the 4 states in the Hibernate Entity Lifecycle?

Answer:

1. Transient State (New):

- Object created with 'new' keyword.
- Not associated with any persistence context or database record.
- Hibernate does NOT track changes.
- Transition to Managed: call em.persist(entity).

2. Managed (Persistent) State:

- Entity is associated with an active EntityManager (persistence context).
- Typically has a corresponding database row.
- Hibernate uses dirty checking — any field change is auto-detected and synced on commit.
- Enter via persist() or find() / JPQL queries.

3. Detached State:

- Previously managed entity that is no longer tracked (EntityManager closed/cleared or detach() called).
- Still exists in memory but changes are NOT auto-saved.
- Reattach using em.merge(entity).

4. Removed State:

- Managed entity marked for deletion using em.remove(entity).
- DELETE SQL is executed on flush/commit.
- After commit, the entity becomes Transient again.

Q22. What is Hibernate Caching? How many levels does it have?

Answer:

Hibernate Caching is a mechanism to reduce the number of database queries by storing frequently accessed objects in memory.

- When an entity is fetched: Hibernate checks cache first → if found, returns from cache (fast). If not → queries database, stores result in cache for future use.
- Benefits: Better performance, fewer DB calls, improved scalability.

Hibernate has 2 levels of caching:

Level 1 Cache (First-Level Cache):

- Enabled by default — no configuration needed.
- Scoped to a single EntityManager instance.
- If the same record is fetched again in the same EM, it's returned from cache without a DB query.
- A different EM instance will NOT share this cache.

Level 2 Cache (Second-Level Cache):

- NOT enabled by default — requires explicit configuration.
- Scoped to EntityManagerFactory — shared across all EntityManagers of one EMF.
- Requires a third-party provider like EhCache.

Section 7: Fetch Types & Relationships (Mappings)

Q23. What are the types of Entity Relationships in Hibernate?

Answer:

1. One-to-One (@OneToOne):

- One record in Table A corresponds to exactly one record in Table B.
- Example: Person ↔ Passport, Student ↔ Laptop.
- Use @OneToOne in both classes for bidirectional.

2. One-to-Many / Many-to-One (@OneToMany / @ManyToOne):

- One record in Table A corresponds to many records in Table B.
- Example: Department → Employees, Bank → Accounts.
- Use @OneToMany in the parent and @ManyToOne in the child.

3. Many-to-Many (@ManyToMany):

- Multiple records in Table A relate to multiple records in Table B.
- Example: Students ↔ Courses, Actors ↔ Movies.
- A junction (join) table is automatically created.

Q24. What is the difference between Unidirectional and Bidirectional mapping?

Answer:

Unidirectional Mapping:

- Navigation is possible in only ONE direction.
- Example: Using a Car object, you can access Engine — but from Engine, you cannot access Car.
- Only one class holds the reference to the other.

Bidirectional Mapping:

- Navigation is possible in BOTH directions.
- Example: From Car, you can access Engine; from Engine, you can access Car.
- Both entity classes hold references to each other.
- Use @OneToOne in both classes.
- By default, creates duplicate foreign key columns — use mappedBy to avoid this.

Q25. What is mappedBy? What is @JoinColumn and @JoinTable?

Answer:

mappedBy:

- Used to avoid duplicate foreign key columns in bidirectional mapping.
- Always placed on the PARENT/non-owning side.
- Compatible with: @OneToOne, @OneToMany, @ManyToMany.
- NOT compatible with @ManyToOne.

```
@OneToOne(mappedBy = "engine")  // in Engine class
```

@JoinColumn:

- Stores the foreign key ID of another entity as a column in the same table.
- Used on the OWNING side of the relationship.
- Better performance — no extra table join needed.

```
@JoinColumn(name = "my_engine_id")
```

@JoinTable:

- Stores IDs of both related entities in a SEPARATE join table.
- Used for Many-to-Many relationships.
- joinColumns → owning entity's FK column; inverseJoinColumns → other entity's FK column.
- More normalized — reduces data redundancy.

Section 8: Scenario-Based & Advanced Questions

Q26. What is the difference between persist() and merge() in Hibernate?

Answer:

persist():

- Used for NEW (Transient) entities.
- Transitions the entity from Transient → Managed state.
- Schedules an INSERT operation.
- Entity must NOT already have an ID set (or the ID must be managed by Hibernate).

merge():

- Used for DETACHED entities — entities that were previously managed but are no longer tracked.
- Copies the state of the detached entity into a new Managed instance.
- Schedules an UPDATE operation.
- Returns the newly managed entity (the original detached object is NOT managed).

Key Rule: Always use the returned object from merge() for further operations.

Q27. What is the difference between hbm2ddl.auto: create, update, and create-drop?

Answer:

The hibernate.hbm2ddl.auto property in persistence.xml controls automatic schema generation:

create:

- Drops and recreates the schema on every application start.
- All existing data is LOST.
- Use during initial development only.

update:

- Updates the existing schema to match the current entity definitions.
- Adds new columns/tables but does NOT remove old ones.
- Existing data is PRESERVED.
- Safe for development and staging environments.

create-drop:

- Creates schema on startup and DROPS it when the application closes.
- Useful for testing — clean state every run.
- All data is lost after each session.

Q28. How do you remove the extra 3rd table in One-to-Many bidirectional mapping?

Answer:
By default, @OneToMany creates a third (junction) table to represent the relationship. This can be eliminated using mappedBy and @JoinColumn.

Steps:

Step 1: In the parent class (e.g., Bank), use mappedBy:

```
@OneToMany(mappedBy = "bank")
private List<Account> accounts;
```

Step 2: In the child class (e.g., Account), use @ManyToOne + @JoinColumn:

```
@ManyToOne
@JoinColumn
private Bank bank;
```

Result: The foreign key (bank_id) is stored directly in the Account table — no separate junction table is created, while bidirectional navigation is maintained.